GOOGLE CLOUD · PROFESSIONAL DATA ENGINEER

The PDE Field Guide

Everything on the current exam guide, explained like a senior engineer over coffee. Less reading, more pattern-spotting.

Built Nov 2026 · aligned to the current 5-section exam guide · fills the gaps in the 2019 notes and the 2022 ExamTopics dump

1	Designing data processing systems	~22%
2	Ingesting and processing the data	~25%
3	Storing the data	~20%
4	Preparing and using data for analysis	~15%
5	Maintaining and automating data workloads	~18%

START HERE

How this exam actually works

Read these 3 pages first. They tell you how to think on exam day, and that matters as much as knowing the products.

Thing	What to know
Format	2 hours · roughly 40–50 questions · single-select and multiple-select · pass/fail (no score is shown)
Where	Online proctored, or a test centre (Pearson VUE). English or Japanese.
Cost / validity	US$200 · valid 2 years · renewal is a shorter 1-hour exam
Question style	Almost every question is a **scenario**: "Company X needs Y with constraint Z. What should you do?" Usually two answers are obviously wrong and two look plausible.
Case studies	Older material (Flowlogistic, MJTelco) is built around case studies. The current guide doesn't list them. Don't memorize those companies, but practise the skill they teach: **read the requirements, then pick the answer that meets every one of them**.
Product names	Google says the exam is being refreshed to match recent rebranding. Learn both old and new names (see the *Old name → New name* translator at the back).

The 6 golden rules of picking a PDE answer

When two answers both "work", Google's preferred one almost always follows these rules. Learn them like free throws.

1. Managed & serverless beats DIY
BigQuery over self-run Hive. Dataflow over custom VMs. Pub/Sub over self-hosted Kafka on VMs. Anything that says "install on Compute Engine" is usually the trap.

2. Least operational overhead wins
Phrases like "minimal management", "small team" or "without managing infrastructure" all point to the serverless option.

3. Meet *every* requirement, then go cheapest
An answer that's cheap but misses one requirement (latency, region, consistency) is wrong. Of the answers that meet all of them, pick the cheapest and simplest.

4. Least privilege, always
Predefined roles at the smallest scope, given to groups or service accounts. Basic roles (Owner/Editor/Viewer) are almost never the right answer.

5. Migration: "lift & shift" first, modernize later
"Migrate existing Spark/Hadoop jobs with minimal changes" → Dataproc. "Rebuild for the cloud / fully managed" → Dataflow / BigQuery.

6. Fix the cause, not the symptom
Slow query? Partition/cluster it before you buy more slots. Hotspotting in Bigtable? Redesign the row key before you add nodes.

EXAM BRAIN · KEYWORD RADAR

Train your eye to circle these words in every question. They decide the answer: **real-time / streaming** (Pub/Sub + Dataflow) · **sub-10 ms, key lookups, time series, IoT** (Bigtable) · **global, strongly consistent, relational** (Spanner) · **analytics, SQL, petabytes** (BigQuery) · **existing Hadoop/Spark** (Dataproc) · **no-code / GUI ETL** (Data Fusion) · **orchestrate, dependencies, retries** (Composer) · **PII, mask, tokenize** (Sensitive Data Protection / DLP) · **exfiltration** (VPC Service Controls) · **own the keys** (CMEK) · **minimize cost** · **minimize ops**.

How to use this guide in 2 weeks

Days	Read	Then do
Day 1–2	Domain 3 – Storing (decision trees first)	Storage quiz set
Day 3–5	Domain 2 – Ingesting & processing (Pub/Sub, Dataflow, Dataproc)	Streaming + pipeline quiz sets
Day 6–7	Domain 1 – Designing (security, migration, reliability)	Security + migration quiz sets
Day 8–9	Domain 4 – Analysis & ML	BigQuery features + ML quiz sets
Day 10	Domain 5 – Maintaining & automating	Ops + cost quiz set
Day 11	Cheat sheet + your wrong answers	**Mock exam #1** (timed, 50 Q / 2 h)
Day 12–13	Re-read only your weak domains	**Mock exam #2**, then #3 if time allows
Day 14	Cheat sheet only. Sleep.	Exam

Domains are ordered by how much they help each other. Storage choices show up in every other domain, so they come first.

The legend

EXAM BRAIN
"If the question says X, the answer is Y." Pattern rules.

TRAP
Wrong answers that look right, and old facts that are no longer true.

PICTURE IT
An analogy so the idea sticks.

YELLOW HIGHLIGHTER
Text highlighted like this is very likely to show up as a question or as the deciding detail in an answer. These are the facts the ExamTopics questions keep coming back to. If you only have 30 minutes, skim the highlights.

BOSS FIGHT
A real-style exam question with the reasoning. Cover the answer and try it first.

TL;DR
End-of-chapter recap. Reread these the night before.

DOMAIN 1

Designing data processing systems

22%

Security, compliance, reliability, portability and migrations. This is the architect's part of the exam.

1.1 Security & compliance

PICTURE IT

Think of GCP security as an office building. **IAM** is the keycard system (who can open which door). **Encryption** locks the filing cabinets. **Sensitive Data Protection (DLP)** blacks out the secret lines in documents. **VPC Service Controls** is the security guard at the exit who stops anyone carrying documents *out* of the building, even people with valid keycards.

IAM: who can do what

- **Hierarchy:** Organization → Folders → Projects → Resources. Permissions are **inherited downward**, and a lower level can't take away what a higher level granted.
- **Principals:** Google accounts (people), **Google Groups** (teams, the preferred way to grant), **service accounts** (apps and pipelines), and Workload Identity Federation (external workloads, no keys).
- **Role types:** *Basic* (Owner/Editor/Viewer, too broad, avoid) · *Predefined* (e.g. `roles/bigquery.dataViewer`, the default choice) · *Custom* (only when no predefined role fits).
- **Service account hygiene:** one SA per pipeline/app, no downloaded keys if you can avoid it, grant `iam.serviceAccountUser` only to people who must "run as" it.
- **Organization policies** are guardrails, not permissions. Examples: `constraints/gcp.resourceLocations` (data residency: only allow EU regions), disable SA key creation, restrict public IPs.

BigQuery role	Can do	Typical exam use
bigquery.dataViewer	Read tables/views	Analysts who only query
bigquery.dataEditor	Read + write table data	Pipelines loading data
bigquery.dataOwner	Edit + manage dataset/table access	Dataset owners
bigquery.jobUser	Run jobs (queries) in a project	**Needed in addition to** dataViewer to actually run queries
bigquery.user	Run jobs + create own datasets	Self-service analysts
bigquery.admin	Everything	Almost never the right answer

TRAP · OLD FACT

The 2019 notes say BigQuery security "cannot be applied at table level". That is **no longer true**. BigQuery now supports table-level IAM, **row-level access policies**, **column-level security (policy tags)** and **dynamic data masking**. Data jobs (running queries) and data access (reading tables) are separate permissions, which is why "dataViewer on the dataset + jobUser on the project" is such a common correct answer.

Encryption & key management

Option	Who holds the key	Pick it when the question says…

Google-managed (default)	Google, automatically	Nothing special. Data is always encrypted at rest and in transit.
CMEK (Cloud KMS)	You manage the key in Cloud KMS; Google uses it	"Control key rotation", "be able to revoke access", "compliance requires we manage keys". The most common answer.
Cloud HSM	CMEK, but keys live in FIPS 140-2 Level 3 hardware	"Hardware security module" requirement
Cloud EKM	Keys stay in an **external** key manager outside Google	"Keys must never be stored in Google Cloud"
CSEK	You supply the raw key with each request	Only Cloud Storage & Compute Engine disks. Rarely the best answer.
Client-side	You encrypt before upload	"Google must never see plaintext"

Crypto-shredding: disable or destroy a CMEK key, and every piece of data encrypted with it becomes unreadable. That's useful for "delete all of a customer's data".

Privacy: PII and Sensitive Data Protection (formerly Cloud DLP)

- **Inspect:** finds 150+ infoTypes (credit cards, emails, ID numbers, custom regex) in BigQuery, Cloud Storage, Datastore, or a data stream via the API.
- **De-identify techniques** (know which keeps what):
  - **Redaction / masking** (`****1234`): irreversible, loses analytics value.
  - **Tokenization / pseudonymization** with **deterministic encryption** or **format-preserving encryption (FPE)**: the same input always gives the same token, so **joins and counts still work**, and it's **reversible** with the key.
  - **Cryptographic hashing** (secure hash): consistent, but one-way.
  - **Bucketing / generalization** (age 34 → 30–39) and **date shifting**: reduce re-identification risk.
- **Risk analysis:** k-anonymity, l-diversity. "Measure the re-identification risk" → DLP risk analysis.
- Typical pipeline: data lands in a GCS staging bucket → Dataflow (or a DLP job) de-identifies it → clean data goes to BigQuery.

> **EXAM BRAIN**
> "Analysts need to join on customer ID but must not see the real ID" → **deterministic tokenization / FPE with DLP**. Not redaction (it destroys the join key) and not hashing alone (you can't reverse it if the business needs to).

Perimeter, network & audit

- **VPC Service Controls (VPC-SC):** draws a perimeter around projects so APIs like BigQuery and GCS can't send data outside it, *even with valid credentials*. Keywords: "prevent data exfiltration", "stolen credentials".
- **Private Google Access:** VMs without public IPs can still reach Google APIs. **Private Service Connect:** private endpoints for Google or partner services.
- **Cloud Audit Logs:** *Admin Activity* (always on, free) · *Data Access* (off by default for most services, **on by default for BigQuery**) · *System Event* · *Policy Denied*. "Who read this table?" → Data Access logs. Export them to BigQuery with a log sink for analysis.
- **Environment separation:** separate **projects** for dev / test / prod (often under folders). Different IAM, different billing visibility, no accidental prod writes.

EXAM BRAIN · DATA RESIDENCY

"Data must stay in the EU" → create resources in EU regions **and** enforce it with the **resource locations org policy**. For BigQuery, the dataset location is fixed at creation; you can't change it later (copy the data to a new dataset instead).

1.2 Reliability & fidelity

"Fidelity" means the data stays correct: cleaned, validated, and not duplicated or lost.

Cleaning & preparing

Dataform: SQL-based (SQLX) transformations inside BigQuery with dependencies, git, and **assertions** (data-quality tests). This is "dbt by Google".

Dataflow: code-based cleaning, batch or stream.

Cloud Data Fusion: GUI drag-and-drop ETL (Wrangler for cleaning).

Gemini in BigQuery: AI-assisted SQL generation and data prep.

Validating

Row counts and checksums between source and target · Dataform assertions · **Dataplex data quality scans** · Dataflow dead-letter outputs for bad records · BigQuery `max_bad_records` only as a quick fix (it silently drops rows).

ACID vs availability, in one table

Need	Choose
Full ACID transactions, regional, up to ~64 TB	Cloud SQL / AlloyDB
Full ACID + global scale + 99.999% availability	Spanner
Single-row atomicity only, massive throughput	Bigtable
Document transactions for apps	Firestore
Multi-statement transactions for analytics	BigQuery (supports them, but it's not an OLTP database)

Disaster recovery vocabulary

- **RPO** (Recovery Point Objective) = how much data you can afford to lose. **RTO** (Recovery Time Objective) = how long you can be down.
- Lower RPO/RTO = more money. Pick the cheapest option that meets the stated numbers.
- Patterns: **multi-region** storage (GCS, BigQuery, Spanner) · **cross-region replicas** (Cloud SQL) · **replication with failover** (Bigtable multi-cluster) · **backups/snapshots** to another region (cheapest, highest RTO).

1.3 Flexibility & portability

- **Multi-cloud analytics: BigQuery Omni** queries data that sits in AWS S3 / Azure Blob *without moving it*. **BigLake** gives one governed table layer over open formats (Parquet, ORC, **Apache Iceberg**) in GCS/S3/Azure, with fine-grained security.
- **Portability:** open formats (Avro, Parquet, Iceberg), open-source engines (Beam runs on Dataflow, Flink or Spark; Spark runs on Dataproc anywhere), containers.
- **Discovery & cataloging: Dataplex Universal Catalog** (formerly Data Catalog + Dataplex): search all your data assets, technical and business metadata, profiling, lineage.

- **Staging:** raw → curated → consumption zones (bronze / silver / gold) in GCS + BigQuery.

1.4 Data migrations

PICTURE IT

Moving house. Small boxes go in your car (**gcloud storage** over the internet). A full house takes a moving company on a schedule (**Storage Transfer Service**). A whole warehouse goes by **shipping container** (**Transfer Appliance**). And if people still live there while you move, you need someone copying every change as it happens (**Datastream / Database Migration Service**).

Tool	Moves what	Choose when...
gcloud storage (gsutil)	Files → GCS	Small/medium one-off copies. Use parallel (`-m`) for many files, and combine small files into bigger ones.
Storage Transfer Service	S3, Azure Blob, HTTP lists, other GCS buckets, **on-prem file systems (with agents)** → GCS	Large, scheduled, recurring, or cross-cloud transfers with good bandwidth
Transfer Appliance	Physical device you fill and ship (hundreds of TB to PB)	Huge data + slow/expensive network. "It would take weeks/months over the network."
BigQuery Data Transfer Service	SaaS (Google Ads, YouTube, etc.), S3, **Teradata, Redshift**, scheduled queries → BigQuery	Data-warehouse migration, recurring SaaS loads
Database Migration Service	MySQL, PostgreSQL, SQL Server, Oracle → Cloud SQL / AlloyDB	Migrate an **operational database** with minimal downtime (continuous replication, then cut over)
Datastream	Serverless **CDC** (change data capture) from MySQL, PostgreSQL, Oracle, SQL Server → BigQuery / GCS	"Replicate changes in near real time into BigQuery for analytics"

EXAM BRAIN · TRANSFER TIME MATH

Roughly: **1 TB over 100 Mbps ≈ 1 day**. 100 TB over 100 Mbps ≈ 3+ months → Transfer Appliance. 100 TB over 10 Gbps ≈ 1 day → Storage Transfer Service. If the question gives you bandwidth and size, do this math. That's what it's testing.

Networking for migrations & hybrid

- **Cloud VPN**: encrypted IPsec over the internet, quick to set up, lower bandwidth.
- **Dedicated Interconnect**: private physical link (10/100 Gbps), highest throughput. **Partner Interconnect**: through a provider, when you can't reach a Google colocation facility.
- "High-throughput, private, consistent connection to on-prem" → Interconnect. "Quick and secure, moderate volume" → VPN.

Migration playbook (how Google wants you to think)

Assess — Inventory current systems, stakeholders, SLAs › **Plan** — Target design, lift-and-shift vs modernize › **Migrate** — Move data + jobs (DTS, DMS, STS, Datastream) › **Validate** — Row counts, checksums, run both in parallel › **Optimize** — Partition, re-architect, decommission

On-prem thing	Lift & shift to	Modernize to
Hadoop / Spark / Hive / Pig	Dataproc (GCS replaces HDFS)	BigQuery + Dataflow / Serverless Spark
Teradata / Redshift / Netezza	BigQuery via BigQuery DTS / migration service	BigQuery with partitioning and nested data
Kafka	Managed Service for Apache Kafka	Pub/Sub
MySQL / PostgreSQL	Cloud SQL (via DMS)	AlloyDB / Spanner if scale demands
HBase / Cassandra	Bigtable (HBase API compatible)	Bigtable
Airflow	Cloud Composer	Cloud Composer
Cron + scripts	Cloud Scheduler + Cloud Run	Composer / Workflows

BOSS FIGHT

A bank must analyse transaction data in BigQuery. Auditors require that the bank can make the data permanently unreadable on demand, and that analysts can join tables on account number without ever seeing real account numbers. What do you do?

- A. Use Google-managed encryption and redact account numbers with DLP
- B. Use CMEK in Cloud KMS, and tokenize account numbers with deterministic encryption via DLP
- C. Use CSEK on BigQuery and hash account numbers
- D. Store data in GCS with Bucket Lock

Answer: B. CMEK lets you revoke or destroy the key (crypto-shredding). Deterministic tokenization keeps joins working and hides the real value. A breaks joins and doesn't give key control. C fails because CSEK isn't supported in BigQuery. D is about retention, not secrecy.

TL;DR · DOMAIN 1

- Predefined roles, to groups or service accounts, at the smallest scope. **dataViewer + jobUser** is the classic analyst combination.
- Control keys → CMEK. Keys outside Google → EKM. Never see plaintext → client-side.
- PII: redact (gone) vs tokenize/FPE (joinable, reversible) vs hash (joinable, one-way).
- Exfiltration → VPC-SC. Residency → region choice + org policy.
- Migrations: STS (online, big) · Appliance (offline, huge) · DTS (warehouses/SaaS → BQ) · DMS (databases) · Datastream (CDC).

DOMAIN 2 · THE BIGGEST ONE

Ingesting & processing the data

25%

Pub/Sub, Dataflow (Apache Beam), Dataproc, Data Fusion, Composer, plus streaming concepts that show up on almost every exam.

2.1 The big picture: the canonical pipelines

Sources	Ingest	Process	Store	Serve
Apps, IoT, logs, DBs (CDC), files	Pub/Sub · Datastream · STS · Kafka	Dataflow · Dataproc · BigQuery SQL · Data Fusion	BigQuery · Bigtable · GCS · Spanner	Looker · BI · ML · APIs

Streaming classic: Pub/Sub → Dataflow → BigQuery (analytics) and/or Bigtable (low-latency serving). **Batch classic:** GCS → Dataflow or Dataproc → BigQuery. Or simply GCS → **BigQuery load job → SQL (ELT)**, which is often the simplest correct answer.

EXAM BRAIN · WHICH PROCESSING ENGINE?

Existing Spark/Hadoop/Hive code, want minimal changes?
→ Dataproc (or Serverless for Apache Spark)

New pipeline, batch *and/or* streaming, want serverless + autoscaling?
→ Dataflow (Apache Beam)

Data is already in BigQuery and the transform is SQL-able?
→ BigQuery SQL / Dataform (ELT)

Team wants no-code, visual pipelines with lots of connectors?
→ Cloud Data Fusion

Business analysts want to visually clean and explore messy files?
→ Dataprep (Alteryx Designer Cloud)

Just react to one event (file landed, message arrived) with small logic?
→ Cloud Run functions (formerly Cloud Functions)

2.2 Pub/Sub, the message post office

PICTURE IT

Pub/Sub is a post office. Publishers drop letters into a **topic** (a mailbox). Every **subscription** is a separate delivery route that gets its **own copy** of every letter. A subscriber must sign for (**ack**) each letter; if it doesn't sign within the **ack deadline**, the post office delivers it again. That's why you get duplicates.

- **Global, serverless, autoscaling.** No partitions or brokers to manage (unlike Kafka).
- **Delivery:** **at-least-once** by default, so duplicates can happen. **Exactly-once delivery** can be enabled on pull subscriptions (within a region).
- **Ordering:** not guaranteed by default. Use **ordering keys** for per-key order (e.g. per customer).

- **Pull vs push:** pull = subscriber asks (high throughput, Dataflow uses this). Push = Pub/Sub sends HTTPS to an endpoint (Cloud Run, webhooks).
- **Export subscriptions: BigQuery subscription** and **Cloud Storage subscription** write messages directly, with **no Dataflow needed** when no transformation is required.
- **Retention:** unacked messages are kept 7 days by default (configurable). Topics can also retain messages (up to 31 days).
- **Replay: Seek** a subscription to a **timestamp** or **snapshot** to reprocess messages (for example after deploying a buggy subscriber).
- **Dead-letter topic:** after N failed delivery attempts, the message goes to a DLQ topic so it doesn't block everything else.
- **Schemas:** enforce Avro/Protobuf message schemas on a topic.
- **Monitoring:** `subscription/num_undelivered_messages` (backlog) and `oldest_unacked_message_age` show consumers falling behind.

TRAPS

- "Lots of duplicate messages" → the subscriber isn't acking within the deadline. Fix: ack faster, extend the ack deadline, or dedupe in Dataflow using a unique ID. Don't just "add more subscribers".
- Two different apps both need **all** messages → **two subscriptions.** Two workers sharing load → one subscription with multiple subscribers.
- Pub/Sub Lite has been shut down. If you see it in old dumps, the modern answer is Pub/Sub or Managed Kafka.

Kafka on GCP

Google Cloud Managed Service for Apache Kafka gives you real Kafka without running brokers. Use it when the company has Kafka clients or apps and wants minimal changes. Dataflow can read Kafka with `KafkaIO`. If they're open to change and want the least ops → Pub/Sub.

2.3 Dataflow & Apache Beam, the star of the exam

Dataflow is Google's serverless runner for **Apache Beam** pipelines. Beam is the code model, and the **same code** works for batch and streaming.

Beam concept	Meaning
Pipeline	The whole job: read → transform → write
PCollection	A distributed dataset. *Bounded* = batch, *unbounded* = stream.
PTransform	A step. `ParDo` (map/filter per element, through a `DoFn`), `GroupByKey`, `CoGroupByKey` (join), `Combine` (sum/avg, efficient), `Flatten` (merge PCollections), `Partition` (split)
Side input	A small extra dataset every worker can read, like a lookup table (e.g. currency rates). "Enrich each event with a small reference table" → side input.
Multiple outputs	Tagged outputs, e.g. valid rows → BigQuery and invalid rows → a **dead-letter** table/GCS.
Runner	Where it runs: DataflowRunner (cloud) or DirectRunner (local testing). Also Flink and Spark.

Time: event time vs processing time

PICTURE IT

You post holiday photos from Bali, but your phone has no signal until you're back in Jakarta. **Event time** = when the photo was taken (Bali, Monday). **Processing time** = when the server received it (Jakarta, Friday). For correct analytics you almost always group by **event time**. The **watermark** is Dataflow's estimate of "I've probably seen everything up to time T".

Windows

Window	Shape	Example question phrase
Fixed (tumbling)	Non-overlapping equal chunks: 10:00–10:05, 10:05–10:10…	"Count orders every 5 minutes"
Sliding (hopping)	Overlapping: a 60-min window starting every 5 min	"Moving average", "last hour, updated every 5 minutes"
Session	Per key, closes after a **gap of inactivity**	"User sessions", "group clicks until the user is idle 30 min"
Global	One window forever (the default)	Needs triggers to emit anything on a stream

Triggers, lateness & accumulation

- **Trigger** = *when* to emit a window's result. Default: when the watermark passes the end of the window, then again for each late element.
- Types: **event-time** (`AfterWatermark`), **processing-time** (`AfterProcessingTime`, e.g. "early results every 1 min"), **data-driven** (`AfterCount`), and **composite** combinations.
- **Allowed lateness**: how long after the watermark you'll still accept late data into a window. Data later than that is dropped.
- **Accumulating** mode: each firing re-emits the full updated total. **Discarding** mode: each firing emits only the new part.

EXAM BRAIN

"Dashboards need speculative early results, then a corrected final value when late data arrives" → **early processing-time trigger + watermark trigger + allowed lateness, accumulating mode**.

Running Dataflow in production

- **Exactly-once processing** inside the pipeline. Pair it with idempotent sinks (or the BigQuery Storage Write API) for end-to-end correctness.
- **Autoscaling** (horizontal and vertical), **Streaming Engine** (moves state off workers), **Dataflow Prime**, and **Dataflow Shuffle** for batch.
- **Templates: Flex Templates** (container-based, the modern choice) let non-developers launch parameterized pipelines. Google provides ready templates, e.g. Pub/Sub → BigQuery, GCS text → BigQuery.
- **Updating a streaming job:** `--update` replaces the job in place and keeps state, as long as the changes are compatible. Incompatible change → **Drain** the old job (finishes in-flight data, no loss) and start a new one. **Cancel** stops immediately and loses in-flight data.
- **Snapshots** save a streaming job's state so you can restart or roll back.
- **Hot keys** (one key with tons of data): use `Combine.perKey().withHotKeyFanout()`, or use Combine instead of GroupByKey.
- **Security:** run workers with a dedicated **worker service account**, use **private IPs** (`--no_use_public_ips`) plus Private Google Access, and set CMEK on the job if required.
- **Roles:** `dataflow.developer` (run and manage jobs), `dataflow.viewer`, `dataflow.worker` (for the worker SA).
- **Monitoring: system lag** and **data freshness** rising means the pipeline is falling behind. Fixes: increase max workers, fix hot keys, remove slow external calls.
- **AI enrichment:** Beam `RunInference` calls models (including Vertex AI endpoints) inside the pipeline, e.g. sentiment scoring on streaming reviews.

TRAP

Calling an external API for every element is slow. Batch the calls, use `GroupIntoBatches`, or use a side input / cache. Writing to BigQuery with legacy streaming inserts costs more and can create duplicates. The **Storage Write API** is the modern default.

2.4 Dataproc: managed Spark & Hadoop

PICTURE IT

Old-school Hadoop is buying a car you keep forever and maintain yourself. Dataproc lets you **rent a car for each trip** (an ephemeral cluster per job), and your luggage stays at home in **Cloud Storage** instead of the car's boot (HDFS). Drop the car off when you're done and stop paying.

- Clusters start in about 90 seconds. Billed **per second** (1-minute minimum).
- **Use GCS, not HDFS**, for persistent data (the Cloud Storage connector: `gs://` instead of `hdfs://`). That separates storage from compute, so clusters can be deleted.
- **Ephemeral (job-scoped) clusters** are the recommended pattern: create, run the job, delete. Use **scheduled deletion** (idle timeout) as a safety net. **Workflow templates** do create → jobs → delete for you.
- **Cost:** add **secondary workers** as **Spot/preemptible VMs** (they only do compute, never hold HDFS data). Keep primary workers standard. Use **autoscaling policies**.
- **Customize: initialization actions** (scripts that install packages on every node at creation), **cluster properties** (config files), custom images, optional components (Jupyter, Flink, Presto/Trino).
- **Dataproc Metastore:** managed Hive metastore shared across ephemeral clusters.
- **Serverless for Apache Spark:** submit a Spark batch with **no cluster at all**. "Run Spark without managing clusters" → this.
- **Component Gateway:** secure web access to YARN, Spark UI and Jupyter (replaces the old SOCKS-proxy approach).
- Connectors: Spark BigQuery connector, Bigtable (HBase API), GCS.

EXAM BRAIN

Persistent cluster only when you need it always on (e.g. interactive, constant workload). Otherwise **ephemeral per job** + GCS + preemptible secondaries. "Our Dataproc costs are high and the cluster sits idle at night" → ephemeral clusters / scheduled deletion / autoscaling.

2.5 Other processing tools

Tool	What it is	Pick it when
Cloud Data Fusion	Visual, no-code ETL on CDAP with 150+ connectors and plugins. Pipelines run on Dataproc.	"ETL developers without coding skills", "many connectors", "hybrid/on-prem sources"
Dataprep	Visual data wrangling (Trifacta/Alteryx). Runs on Dataflow/BigQuery.	"Analysts explore and clean data visually"
Dataform	SQLX workflows inside BigQuery: dependencies, assertions, git, scheduling	"ELT in SQL with version control and testing"
BigQuery SQL	Scheduled queries, procedures, MERGE, **continuous queries** (SQL on streaming data, out to Pub/Sub/Bigtable)	Data already in BQ
Cloud Run / Run functions	Event-driven code	"When a file lands in GCS, kick off X" (via Eventarc)

2.6 Deploying & operationalizing pipelines

Orchestrator	Best for
Cloud Composer	Managed **Apache Airflow**. Complex DAGs across many services with dependencies, retries, backfills, SLAs, and hundreds of operators (BigQuery, Dataflow, Dataproc, GCS sensors). DAG files live in the environment's GCS `dags/` folder.
Workflows	Serverless, lightweight orchestration of **HTTP/API calls** and Google services in YAML. Cheaper and simpler than Composer for short service chains.
Cloud Scheduler	Managed cron. Triggers one thing on a schedule (HTTP, Pub/Sub, Workflows).
BigQuery scheduled queries / Dataform schedules	Pure-SQL recurring jobs
Eventarc	Event-driven triggers (e.g. a GCS object finalized → start Cloud Run / Workflows)

CI/CD: code in git → **Cloud Build** runs tests → images to **Artifact Registry** → deploy (Dataflow Flex Template, Composer DAG sync to the bucket, Dataform release). Infrastructure as code: **Terraform**. Keep dev/test/prod in separate projects and promote the same artifact through them.

EXAM BRAIN

"Run Dataproc job, then a BigQuery load, then email if it fails, every night" → **Composer**. "Call three APIs in sequence, serverless, cheap" → **Workflows**. "Run one query at 2 am" → **scheduled query**.

BOSS FIGHT

Clickstream events arrive through Pub/Sub. Mobile users may be offline, so events can arrive up to 2 hours late. You need per-user session metrics in BigQuery that are correct even with late events, with early results visible to dashboards. What do you build?

Answer: Dataflow streaming pipeline, using **event-time timestamps**, **session windows** keyed by user, **allowed lateness of 2 hours**, an early processing-time trigger plus a watermark trigger, in **accumulating** mode, writing to BigQuery via the Storage Write API. Not processing-time windows (wrong sessions for late data) and not a BigQuery subscription (no windowing logic).

TL;DR · DOMAIN 2

- Pub/Sub: at-least-once, duplicates come from slow acks, seek to replay, dead-letter topic, BigQuery subscription for no-transform loads.
- Dataflow: event time + watermark + windows (fixed / sliding / session) + triggers + allowed lateness. Drain to stop safely, update to deploy.
- Dataproc: ephemeral clusters + GCS + preemptible secondary workers. Serverless Spark when you want zero clusters.
- Data Fusion = no-code ETL. Dataprep = visual cleaning. Dataform = SQL ELT with tests.
- Composer = complex DAGs. Workflows = light API chains. Scheduler = cron.

DOMAIN 3 · READ THIS ONE FIRST

Storing the data

20%

Picking the right database is the most tested skill on the PDE. Once this page clicks, a third of the exam gets easier.

3.1 The storage decision tree

Is it files / objects / images / raw data lake / backups?
→ **Cloud Storage**

Is it analytics: SQL over large data, dashboards, a data warehouse?
→ **BigQuery** (open-format lake tables → **BigLake**)

Is it transactional (OLTP) and relational?
→ Regional, up to ~64 TB, standard MySQL/Postgres/SQL Server → **Cloud SQL**
→ PostgreSQL that needs high performance + fast analytics on the same data → **AlloyDB**
→ Global / horizontally scalable / 99.999% / strong consistency → **Spanner**

Is it NoSQL with huge throughput, single-key lookups, time series or IoT, at TB to PB scale, <10 ms latency?
→ **Bigtable**

Is it documents for a web/mobile app, flexible schema, offline sync?
→ **Firestore**

Is it a sub-millisecond cache, session store or leaderboard?
→ **Memorystore** (Redis / Valkey / Memcached)

Service	Type	Scale	Latency	Consistency / Tx	Classic use
Cloud Storage	Object	Unlimited	ms	Strong (objects)	Lake, archive, staging
BigQuery	Columnar warehouse	PB	seconds	ACID per statement	Analytics, BI, ML
Cloud SQL	Relational	≤ ~64 TB, vertical	ms	Full ACID	Web app backends
AlloyDB	PostgreSQL+	Larger, read pools	ms	Full ACID	Heavy Postgres, HTAP
Spanner	Relational, distributed	PB, horizontal	ms	Global strong (external) consistency	Banking, inventory, global games
Bigtable	Wide-column NoSQL	PB, horizontal	<10 ms	Single-row atomic	IoT, time series, ad tech, fintech
Firestore	Document NoSQL	TB	ms	ACID transactions	Mobile/web apps, user profiles
Memorystore	In-memory KV	GBs–TBs	sub-ms	—	Cache, sessions, leaderboards

CLASSIC TRAPS

- **BigQuery is not an OLTP database.** "Low-latency single-row lookups for an app" is never BigQuery.
- **Bigtable has no SQL joins or secondary indexes.** It's searchable by row key only (row-key prefixes and ranges are fine). "Search by many different attributes" → not Bigtable.
- **Bigtable is overkill for small data** (well under 1 TB). It's priced and tuned for scale.
- **Cloud SQL doesn't scale horizontally for writes.** "Writes outgrowing one machine, global users" → Spanner.
- "Datastore" in old dumps = **Firestore in Datastore mode** today.

3.2 Cloud Storage (GCS)

Class	Min. storage duration	Access pattern
Standard	none	Hot data, frequently accessed
Nearline	30 days	About once a month (backups)
Coldline	90 days	About once a quarter
Archive	365 days	Less than once a year (legal/compliance)

All classes have **millisecond access** (no "thaw" delay like tape). Colder classes cost less to store but more to *retrieve*.

- **Lifecycle rules:** automatically change class or delete by age, e.g. "move to Coldline after 90 days, delete after 7 years". **Autoclass** moves objects between classes automatically based on actual access.
- **Location types: region** (lowest latency to compute in that region, cheapest), **dual-region** (two specific regions, HA; **turbo replication** gives a 15-min RPO), **multi-region** (US/EU/ASIA, highest availability).
- **Retention policy + Bucket Lock** = WORM (write once, read many): objects can't be deleted before the retention period ends, *even by admins*. Keywords: "regulators", "must not be deleted for 7 years".
- **Object versioning** keeps old versions. **Soft delete** keeps deleted objects for a window (7 days default) for recovery.
- **Access:** uniform bucket-level access (IAM only, recommended), **signed URLs** (temporary access for someone without a Google account).
- **Performance:** avoid sequential object names at very high request rates. Combine many tiny files into fewer large ones.

3.3 BigQuery storage design

PICTURE IT

BigQuery (on-demand) is like a library that charges you for **every shelf you walk past**, not for the books you take. **Partitioning** splits the library into rooms by date, so you only enter the rooms you need. **Clustering** sorts the books on each shelf, so you stop walking once you've passed what you need. **Selecting fewer columns** = visiting fewer aisles, because storage is columnar.

Partitioning

- Types: **time-unit column** (DATE / TIMESTAMP / DATETIME, by hour, day, month or year) · **ingestion time** (`_PARTITIONTIME` / `_PARTITIONDATE`) · **integer range**.
- Set **require partition filter** so nobody accidentally scans the whole table.
- **Partition expiration** automatically deletes old partitions (e.g. keep 90 days).
- There's a cap on partitions per table (thousands). Don't partition by hour over many years. Use day or month.
- You can't turn an existing table into a partitioned one in place. Use `CREATE TABLE … PARTITION BY … AS SELECT`.

Clustering

- Sorts data by **up to 4 columns**. Great for filters and aggregations on high-cardinality columns (customer_id, country).
- Works with or without partitioning. BigQuery re-clusters automatically for free.

- "Partition by date, cluster by customer_id" is the classic correct answer.

Schema design

- **Denormalize** for analytics. Use **nested and repeated fields** (STRUCT / ARRAY) instead of big joins, e.g. an `orders` table with a repeated `line_items` field.
- Star schemas also work fine. Small dimension tables joined to large fact tables are OK.
- **Avoid:** self-joins on huge tables (use window functions), single-row DML at high frequency (batch it), wildcard `SELECT *`.
- **Sharded tables** (`events_20240101`, `events_20240102`...) are the old way. **Partitioned tables beat date-sharded tables.**

Cost & lifecycle features

Feature	What it does
Long-term storage	Table/partition not modified for 90 days → about 50% cheaper storage, automatically
Time travel	Query or restore data as of any point in the last 2–7 days (`FOR SYSTEM_TIME AS OF`). Then 7 more days of fail-safe (via Google support).
Table snapshots	Cheap, read-only point-in-time copy (you pay only for changed data). Good for backups beyond time travel.
Table clones	Writable lightweight copy, e.g. for testing
Table/dataset expiration	Auto-delete, e.g. temporary staging tables
Physical vs logical billing	Dataset storage billing model. Physical (compressed) can be much cheaper for highly compressible data.

Loading data into BigQuery

- **Batch load jobs are free** (shared slots). Formats from best to worst: **Avro** (self-describing, compressed blocks, parallel), **Parquet/ORC** (columnar), then CSV/JSON.
- **Streaming:** **Storage Write API** (recommended: cheaper, exactly-once with committed streams) vs legacy `insertAll`. Or a Pub/Sub BigQuery subscription.
- **External tables** (query in place): GCS, Google Drive/Sheets, Bigtable, plus **federated queries** to Cloud SQL / Spanner / AlloyDB with `EXTERNAL_QUERY`. Slower than native, but no copy is needed.
- **BigLake tables:** external tables with fine-grained security (row/column), caching, and Iceberg support. **Object tables**: query unstructured files (images, PDFs) in GCS, then feed them to ML.

3.4 Bigtable deep dive

PICTURE IT

Bigtable is a **giant phone book** sorted alphabetically by one thing only: the **row key**. Finding "Santoso" is instant. Finding everyone who lives on a certain street means reading the whole book. Design the row key around **how you'll look things up**.

- **Model:** table → rows (one row key) → **column families** → columns → cells with **timestamps** (versions). **Sparse**: empty cells cost nothing.
- **Only one index: the row key**, sorted lexicographically. Supports point lookups, **prefix** scans and **range** scans.
- Atomic only within **a single row**, so put things that change together in one row.
- Throughput scales **linearly with nodes**. Supports **autoscaling**.
- **Storage type:** SSD (default, low latency) or HDD (cheaper, large mostly-batch archives). **You can't switch**; you'd have to create a new instance and copy the data (e.g. with Dataflow, or export/import).
- APIs: **HBase-compatible** (easy migration from HBase), plus a GoogleSQL interface for queries.

Row key design: the most-tested Bigtable topic

Do	Don't
Put the thing you query by first: `device#sensor#reverse_ts`	Start the key with a **timestamp** (all writes hit one node → **hotspot**)
Field promotion: move fields into the key, e.g. `user_id#timestamp`	Use sequential IDs (1, 2, 3…) as the key prefix
Salting (hash prefix) only if field promotion isn't enough	Use a hashed key if you need range scans (hashing destroys order)
Reverse timestamp (e.g. `MAX_LONG - ts`) **when you mostly want the *newest* data first**	Make keys huge (keep them short; long keys waste memory)
Tall & narrow tables for time series (a new row per event)	Store an unbounded number of versions in one row (rows have a size limit)

EXAM BRAIN

"Bigtable performance is uneven, one node is overloaded" → **hotspotting from a bad row key**. Diagnose with **Key Visualizer**, fix by redesigning the key. Adding nodes doesn't fix a hotspot.

Replication & app profiles

- An instance can have **multiple clusters in different zones/regions**, with automatic replication (eventually consistent between clusters).
- **App profiles** control routing: **multi-cluster routing** (automatic failover, HA, eventual consistency) vs **single-cluster routing** (read-your-writes, and isolates workloads).
- Classic pattern: one cluster serves the low-latency app, another serves heavy batch analytics, each with its own app profile, so batch doesn't slow the app.
- Performance tips: test with at least ~300 GB for a realistic benchmark, and keep the client (GKE/Compute) in the same zone or region.
- Deletes and updates write new data and get cleaned up later by **compaction** and **garbage-collection policies** (max versions / max age per column family).

3.5 Spanner, Cloud SQL, AlloyDB, Firestore, Memorystore

Spanner

- Relational + SQL + ACID + **horizontal scale** + **global** strong consistency (TrueTime).
- 99.99% regional, **99.999% multi-region** SLA.
- Scale in nodes / processing units. Autoscaling available.
- **Primary key trap:** monotonically increasing keys (timestamps, sequences) cause hotspots. Use **UUIDv4** or bit-reversed sequences.
- **Interleaved tables** store child rows next to their parent (fast parent-child joins).
- **Data Boost**: run analytics/BigQuery federated queries without hurting OLTP.

Cloud SQL

- Managed MySQL, PostgreSQL, SQL Server. Scales up (bigger machine), not out.
- **HA** = regional instance with a standby in another zone (automatic failover).
- **Read replicas** offload reads. **Cross-region replicas** for DR or reads near users.
- Automated backups + point-in-time recovery (binary logs / WAL).
- Connect securely via the Cloud SQL Auth Proxy / private IP.

AlloyDB for PostgreSQL

- PostgreSQL-compatible, much faster transactional performance than standard Postgres.
- Built-in **columnar engine** → fast analytical queries on operational data (HTAP).
- Read pool instances, high availability, vector search (AlloyDB AI).
- "Postgres compatibility + better performance than Cloud SQL, without Spanner's redesign" → AlloyDB.

Firestore & Memorystore

- **Firestore**: serverless document DB, automatic indexes on every field, ACID transactions, real-time listeners, offline mobile sync. Native mode vs Datastore mode.
- Can query by properties (unlike Bigtable), but avoid it for heavy analytics → export to BigQuery.
- **Memorystore**: managed Redis / Valkey / Memcached. Sub-ms. "Cache frequently read query results", "session state", "real-time leaderboard".

3.6 Data warehouse planning

- Start from **business questions and access patterns**: who queries what, how often, how fresh does it need to be?
- **Normalize** (3NF) for OLTP integrity. **Denormalize** (star schema, nested/repeated) for analytics speed. BigQuery leans toward denormalized.
- **Slowly changing dimensions:** Type 1 overwrites. Type 2 keeps history with `valid_from` / `valid_to` / `is_current`. Use `MERGE` for upserts.
- Design datasets by **domain and access boundary**. IAM is easiest at the dataset level, e.g. `finance_raw`, `finance_curated`, `marketing_mart`.

3.7 Data lake & data platform: Dataplex

PICTURE IT

A data lake without governance is a "data swamp": lots of files that nobody can find or trust. **Dataplex** is the librarian and the health inspector together. It knows what's in every bucket and table, who owns it, what quality it is, and where it came from.

- **Lake → Zones → Assets**: a lake per domain (e.g. Sales), **raw zones** and **curated zones**, with assets = GCS buckets and BigQuery datasets attached. You manage security and policies centrally while the data stays where it is.
- **Dataplex Universal Catalog** (formerly Data Catalog): search and discovery, technical + business metadata (aspects/tags), glossaries.
- **Data profiling** (column stats) and **data quality scans** (rules like not-null, ranges, uniqueness, with alerts).
- **Data lineage**: tracks how tables were produced (BigQuery, Dataflow, Composer, Dataproc).
- **Federated governance / data mesh**: each domain team owns its data products, while central policies (security, quality, catalog) still apply. That's Dataplex + BigQuery + Analytics Hub (BigQuery sharing).
- **Lake cost controls:** GCS lifecycle rules + Autoclass, BigQuery partition expiration, avoid many small files, monitor with Cloud Monitoring and billing exports.

BOSS FIGHT

An IoT company collects readings from 500,000 sensors every second. The app must show the latest readings for one sensor in under 10 ms, and data scientists run heavy daily scans over months of history. What storage design fits?

Answer: Bigtable with row key `sensor_id#reverse_timestamp` (spreads writes, newest first). Use two clusters with **separate app profiles**: single-cluster routing for the app and another cluster for batch analytics, or export to BigQuery for the scientists. Not BigQuery alone (not <10 ms point reads), not Cloud SQL (write scale), and not a timestamp-first key (hotspot).

TL;DR · DOMAIN 3

- Files → GCS · analytics → BigQuery · OLTP regional → Cloud SQL · fast Postgres → AlloyDB · global OLTP → Spanner · key/value at scale → Bigtable · app docs → Firestore · cache → Memorystore.
- GCS classes: 0 / 30 / 90 / 365-day minimums. Lifecycle rules, Bucket Lock = WORM.
- BigQuery: partition by date + cluster by high-cardinality filters, nested/repeated fields, require partition filter, Avro for loads, Storage Write API for streaming.
- Bigtable: row key = everything. No timestamp-first keys, field promotion before salting, Key Visualizer, app profiles.
- Spanner: no monotonically increasing primary keys. Dataplex = lakes/zones/assets + catalog + quality + lineage.

DOMAIN 4

Preparing & using data for analysis

15%

Making data fast for BI, safe to share, and ready for ML and GenAI.

4.1 Preparing data for visualization

Connecting tools

- **Looker**: enterprise BI with a semantic layer (**LookML**), so metrics are defined once and governed. "Consistent metric definitions across the company" → Looker.
- **Looker Studio** (formerly Data Studio): free, quick dashboards. Also **Connected Sheets** (Google Sheets over BigQuery, billions of rows), Tableau / Power BI via connectors.
- Looker Studio has data **freshness/caching** settings. "Dashboard doesn't show the last hour of data" → adjust the data freshness or refresh the data source cache (the BigQuery table itself is fine).

Making dashboards fast (and cheap)

Feature	What it does	Pick when
BI Engine	In-memory acceleration for BigQuery (reserved memory per project)	"Sub-second dashboard response", "many concurrent BI users"
Materialized views	Precomputed, **automatically refreshed** aggregates. Queries on the base table can be **rewritten automatically** to use them.	"Same aggregation queried repeatedly", "reduce cost of repeated GROUP BY"
Logical views	Saved SQL, no stored data, computed every time	Reuse logic or restrict columns (not for speed)
Precalculated fields / summary tables	Scheduled query or Dataform model writing aggregates	Complex logic that materialized views don't support
Query cache	Identical query within 24 h on unchanged tables → free and instant	Automatic (disabled by non-deterministic functions like `CURRENT_TIMESTAMP()`)
Search indexes	`SEARCH()` function for needle-in-haystack text/ID lookups	"Find specific log entries / IDs in huge tables"

Troubleshooting slow or expensive queries

1. Look at the **query execution plan / execution graph** (stages, slot time, bytes shuffled, skew).
2. Select only the columns you need (never `SELECT *`). `LIMIT` **doesn't reduce bytes scanned** on normal tables.
3. Filter on **partition** and **cluster** columns early.
4. Join big to small, and filter *before* the join. Watch for **data skew** (one key with most rows) and **cross joins**.
5. Replace self-joins with **window functions**. Use `APPROX_COUNT_DISTINCT` when exact isn't required.
6. Use a **dry run** to see bytes scanned before running. Set **maximum bytes billed** to cap mistakes.

Security when exposing data

Need	Feature

Share query results without access to the source tables	**Authorized view** (view in a separate dataset, authorized on the source dataset). Also authorized datasets and authorized routines.
Different users see different rows (e.g. only their country)	**Row-level access policies** (`CREATE ROW ACCESS POLICY … FILTER USING (region = 'ID')`)
Hide sensitive columns from some users	**Column-level security** with **policy tags** (taxonomy in Dataplex/Data Catalog). Grant Fine-Grained Reader to those allowed.
Users can query the column but see it masked	**Dynamic data masking** (hash, nullify, last 4 characters...) through policy tags + data policies
Find and transform PII before it lands	Sensitive Data Protection (DLP)

4.2 Preparing data for AI & ML

ML fundamentals the exam still asks about

The old dumps are full of these. They're cheap points once you know the patterns.

Symptom	Meaning	Fix
Great on training data, bad on new data	**Overfitting**	**Regularization** (L1/L2), **dropout**, more training data, fewer features, early stopping, simpler model
Bad on training *and* test data	**Underfitting**	More features / feature crosses, more complex model, train longer
Great offline, bad in production	**Training-serving skew** or data drift	Same preprocessing in training and serving (BigQuery ML `TRANSFORM`, Vertex AI Feature Store), monitor drift
Suspiciously perfect accuracy	**Data leakage** (target info in the features)	Remove leaking features, split data by time
Loss jumps around / diverges	Learning rate too high	Lower the learning rate
Training is very slow to converge	Learning rate too low / unscaled features	Raise the learning rate, normalize features

- **L1** pushes weights to exactly 0 (feature selection, sparse models). **L2** shrinks all weights (general overfitting fix).
- **Precision** = of the things I flagged, how many were right (care about false alarms). **Recall** = of all the real positives, how many I caught (care about misses, e.g. fraud or cancer). **F1** balances both. **AUC-ROC** measures ranking quality across thresholds.
- **Imbalanced classes** (1% fraud): accuracy is misleading. Use precision/recall/PR-AUC, and downsample the majority class or upweight the minority.
- **Feature engineering:** one-hot encoding for low-cardinality categories · **embeddings** or **hash buckets** for high-cardinality ones (user IDs, words) · **bucketization** for numbers with nonlinear effects (age bands) · **feature crosses** (lat × long) · normalize/standardize numeric features.
- **Data splits:** train / validation (tuning) / test (final check). For time series, split by time, never randomly.
- **Model types:** regression (a number), classification (a category), clustering (unsupervised groups), recommendation (matrix factorization), forecasting (time series). Wide & deep models: wide = memorization, deep = generalization.
- **Continuously changing behaviour** (fashion, fraud): retrain regularly on a **combination of old and new data**, and evaluate on recent data.

BigQuery ML: ML with SQL

```
CREATE MODEL mydataset.churn_model
OPTIONS(model_type='logistic_reg', input_label_cols=['churned']) AS
SELECT * EXCEPT(customer_id) FROM mydataset.training_data;

SELECT * FROM ML.EVALUATE(MODEL mydataset.churn_model);
SELECT * FROM ML.PREDICT(MODEL mydataset.churn_model, TABLE mydataset.new_customers);
```

Problem	BigQuery ML model_type
Predict a number (price, revenue)	`linear_reg`, `boosted_tree_regressor`, `dnn_regressor`

Predict a category (churn yes/no)	`logistic_reg`, `boosted_tree_classifier`, `dnn_classifier`, AutoML
Customer segmentation (no labels)	`kmeans`
Product recommendations	`matrix_factorization`
Forecast sales / anomalies in time series	`arima_plus` (also `ML.DETECT_ANOMALIES`)
Use an existing TensorFlow/ONNX model	Imported models
Call Gemini / Vertex AI models from SQL	**Remote models** (`ML.GENERATE_TEXT`, `ML.GENERATE_EMBEDDING`)

Use `TRANSFORM` in `CREATE MODEL` so the same preprocessing runs automatically at prediction time (avoids skew). `ML.EXPLAIN_PREDICT` / `ML.GLOBAL_EXPLAIN` give explainability. Models can be exported to Vertex AI for online serving.

EXAM BRAIN · WHICH ML TOOL?

Data analysts, data already in BigQuery, SQL skills → BigQuery ML.
Common task, no ML expertise (vision, text, speech, translation, documents) → **pre-trained APIs**: Vision, Natural Language, Speech-to-Text, Translation, Video Intelligence, **Document AI**, or Gemini.
Custom model, little ML code, labelled data → **Vertex AI AutoML**.
Full control, data scientists, TensorFlow/PyTorch → **Vertex AI custom training** (GPUs/TPUs, hyperparameter tuning).
Repeatable ML workflow → **Vertex AI Pipelines**. Notebooks → **Vertex AI Workbench / Colab Enterprise** (Datalab is gone).

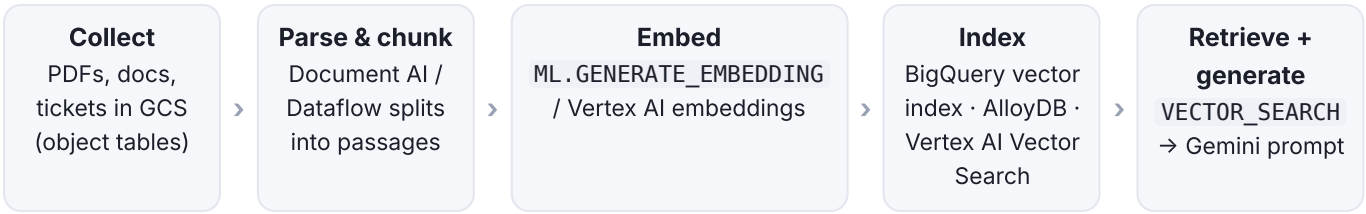
Vertex AI essentials

- **Training:** AutoML or custom jobs. Distributed training uses workers + parameter servers / reduction servers. GPUs for deep learning, **TPUs** for very large TensorFlow/JAX models.
- **Feature Store:** share features and serve them consistently online and offline (prevents training-serving skew).
- **Model Registry → Endpoints:** **online prediction** (low latency, synchronous) vs **batch prediction** (large volumes, asynchronous, output to GCS/BigQuery).
- **Model Monitoring:** detects training-serving skew and prediction drift.

GenAI: embeddings, vector search & RAG NEW TOPIC

PICTURE IT

An **embedding** turns text (or an image) into a list of numbers, a point on a "meaning map". "Cheap flight to Bali" and "budget airfare Denpasar" land right next to each other even though they share no words. **Vector search** finds the nearest points. **RAG** (Retrieval-Augmented Generation) = find the most relevant chunks of your own documents, then hand them to the LLM so it answers from *your* data instead of guessing.



Data-engineering concerns the exam cares about: chunk size and overlap, keeping embeddings fresh when documents change (re-embed incrementally), metadata for filtering, and the same access controls on retrieved data as on the source.

4.3 Sharing data

- **Analytics Hub** (now branded **BigQuery sharing**): publishers create **exchanges** and **listings**; subscribers get a **linked dataset**, a read-only pointer with **no data copy**. Data stays fresh and the subscriber pays for their own queries. Works across orgs. Public, private or commercial exchanges.
- **Data clean rooms** (built on Analytics Hub): two companies analyse combined data with privacy rules (aggregation thresholds) without seeing each other's raw rows.
- Within an org: dataset IAM, authorized views, or Analytics Hub when there are many consumers or teams.
- **Publishing reports:** Looker/Looker Studio sharing, scheduled deliveries, embedding.
- Cross-project rule: the project that **runs the query pays** for compute (on-demand); the project that owns the table pays for storage.

EXAM BRAIN

"Share a dataset with external partners, keep it up to date, no copies, partners pay for their own queries" → **Analytics Hub / BigQuery sharing**. Not "export to GCS and send a signed URL", and not "copy the dataset to their project every night".

BOSS FIGHT

Regional sales managers in Looker Studio must see only their own region's rows. The finance team may see the `margin` column, everyone else sees it nullified. Minimal maintenance. What do you do?

Answer: A BigQuery **row-level access policy** per region (granted to each region's group), plus a **policy tag** on `margin` with a **dynamic data masking** rule (nullify) for everyone, and Fine-Grained Reader for the finance group. Not one view per region (painful to maintain) and not separate tables (data duplication).

TL;DR · DOMAIN 4

- Fast BI: BI Engine (in-memory), materialized views (auto-refresh, auto-rewrite), partitions and clusters. LIMIT doesn't save money.
- Security: authorized views (results only) · row access policies (rows) · policy tags (columns) · dynamic masking (masked values).
- Overfitting → regularization / dropout / more data. Imbalanced data → precision/recall, not accuracy.
- BigQuery ML for SQL users · pre-trained APIs for common tasks · AutoML for custom with little code · Vertex custom training for full control.
- RAG = chunk → embed → vector index → retrieve → Gemini. Sharing without copies → Analytics Hub.

DOMAIN 5

Maintaining & automating data workloads 18%

Cost, capacity, scheduling, monitoring and failure. This is the "day 2" job, and many candidates under-prepare for it.

5.1 Optimizing resources & cost

BigQuery pricing models

Model	How you pay	Good for
On-demand	Per TiB of data scanned (first 1 TiB/month free). A project gets a shared pool of up to ~2,000 slots.	Spiky, unpredictable or small workloads
Capacity (Editions)	Pay for **slots** (compute) per hour: **Standard**, **Enterprise**, **Enterprise Plus** editions. **Reservations** with a **baseline** + **autoscaling** up to a max. Optional 1- or 3-year **commitments** for discounts.	Steady heavy workloads, predictable budgets, workload isolation

TRAP · OLD FACT

"Flat-rate pricing" in older material has been replaced by **BigQuery Editions**. If a dump answer says "buy flat-rate slots", the modern equivalent is "create a reservation with Editions (baseline + autoscaling, optionally a commitment)".

- **Reservations & assignments:** create reservations (e.g. `etl`, `bi`, `adhoc`) and **assign** projects/folders/orgs to them, so heavy ETL can't starve dashboards. Idle slots can be shared between reservations.
- **Edition features:** Enterprise adds features like BigQuery ML in reservations, security features, BI Engine with reservations. Enterprise Plus adds things like **managed disaster recovery** (cross-region failover) and stronger compliance.
- **Cost controls:** **custom quotas** (max bytes per user/project per day) · **maximum bytes billed** on a query · dry runs · budgets and alerts in Cloud Billing · billing export to BigQuery for analysis.
- **Storage:** long-term pricing kicks in automatically after 90 days with no edits. Partition expiration and table expiration for temporary data.

Other cost levers

- **Dataproc:** ephemeral clusters, Spot secondary workers, autoscaling, scheduled deletion. **Persistent clusters** only for constant or interactive use.
- **Dataflow:** autoscaling with a max-workers cap, **FlexRS** (flexible resource scheduling: cheaper batch jobs that can wait), right-sized machine types, Streaming Engine.
- **GCS:** lifecycle rules, Autoclass, the right location type, avoid frequent reads from Archive/Coldline.
- **Bigtable / Spanner:** autoscaling, HDD for cold big data (Bigtable), don't over-provision.
- **Ensure critical jobs have resources:** dedicated reservations (BigQuery), separate Bigtable clusters/app profiles, Dataflow max workers, priority queues in Composer.

5.2 Automation & repeatability

- **Cloud Composer (Airflow) DAGs:** tasks as operators (`BigQueryInsertJobOperator`, `DataflowStartFlexTemplateOperator`, `DataprocSubmitJobOperator`), **sensors** wait for conditions (e.g. a

file in GCS), **retries**, **backfills** (catchup), and **SLAs** and alerts. DAG code goes into the environment's GCS bucket.

- **Idempotency:** rerunning a task must give the same result. Use `WRITE_TRUNCATE` on a partition (e.g. `table$20261104`) or `MERGE` instead of blind appends.
- **Parameterize** by execution date (`{{ ds }}`) so backfills and reruns work.
- Keep DAGs light. Do heavy work in BigQuery/Dataflow/Dataproc, not on Composer workers.
- **Composer environment health:** scale workers and schedulers. "Tasks stuck in queued" → not enough worker capacity or concurrency settings too low.

5.3 Organizing workloads by business need

- **Interactive queries** (default): run as soon as possible, count toward concurrency.
- **Batch queries**: queued and started when resources are free. They don't count toward the concurrent-query limit. "Non-urgent reports that can wait" → batch priority.
- **Workload isolation:** separate reservations for ETL vs BI vs data science. Separate projects for billing visibility.
- Capacity planning: watch slot usage over time (`INFORMATION_SCHEMA.JOBS`, admin resource charts) to size the baseline vs autoscaling.

5.4 Monitoring & troubleshooting

Tool	Use it to...
Cloud Monitoring	Metrics, dashboards and **alerting policies**: Dataflow system lag / data freshness, Pub/Sub backlog and oldest unacked message age, BigQuery slot usage, Bigtable CPU and latency, Composer task failures
Cloud Logging	Logs from every service. **Log-based metrics** (alert on specific error text). **Log sinks** route logs to BigQuery (analysis), GCS (archive) or Pub/Sub (streaming to SIEM).
Cloud Audit Logs	"Who did what, when": admin changes, data access (BigQuery data access is logged by default)
BigQuery admin resource charts & `INFORMATION_SCHEMA.JOBS`	Slot consumption, the most expensive queries, who ran what, errors, queueing
Error Reporting / Cloud Trace / Profiler	App exceptions, request latency, CPU hotspots in code
Dataflow job graph & Dataproc UIs	Find the slow step, stragglers, hot keys, OOM errors, executor failures
Cloud Billing reports & billing export	"Why did costs jump?" → billing export to BigQuery + labels per team/job

EXAM BRAIN · COMMON ERRORS

quotaExceeded / rateLimitExceeded / 429 / 503 → retry with **exponential backoff** (and request a quota increase if it's sustained).
Resources exceeded in BigQuery → the query is too big for one stage: reduce data early, avoid `ORDER BY` without `LIMIT` on huge results, break up the query.
Dataflow workers out of memory → larger machine type, fix hot keys, avoid huge side inputs or GroupByKey on one giant key.
Pub/Sub backlog growing → the subscriber is too slow: scale the consumers (Dataflow max workers), or fix ack handling.
Permission denied → check the *service account* the job runs as, not your own user.

5.5 Designing for failure

Service	Failure & recovery tools
BigQuery	Multi-region datasets (US/EU) · **time travel** (up to 7 days) + fail-safe · table snapshots · **cross-region dataset replication** · **managed disaster recovery** with failover reservations (Enterprise Plus)
Cloud Storage	Dual-/multi-region buckets · turbo replication (15 min RPO) · object versioning · soft delete
Pub/Sub	Messages persisted across zones · **seek to snapshot/timestamp** to replay · dead-letter topics
Dataflow	Automatic worker retries · **drain** and **update** · job **snapshots** · run in a different region when a region fails · dead-letter outputs for corrupt records
Dataproc	Data in GCS (survives cluster loss) · **high-availability mode** (3 masters) · just recreate ephemeral clusters
Bigtable	Multi-cluster replication + multi-cluster routing (automatic failover) · backups

Spanner	Multi-region configs (99.999%) · point-in-time recovery · backups
Cloud SQL	HA (regional standby) for zone failures · **cross-region read replica** promoted for region failure · PITR backups
Composer	Retries per task · snapshots of the environment · DAGs in git

Corrupt or missing data: validate at ingest (schemas, Dataplex quality scans, Dataform assertions), route bad records to a dead-letter location instead of failing the whole job, keep raw immutable copies in GCS so you can **reprocess**, and design pipelines to be idempotent so replays don't create duplicates.

BOSS FIGHT

Your company's nightly ETL in BigQuery is delaying executive dashboards every morning, and the monthly on-demand bill is high and unpredictable. What should you do?

Answer: Move to **BigQuery Editions** with **separate reservations** for ETL and BI (assign each project to its reservation), set a baseline that fits the steady load plus autoscaling for peaks, and consider a commitment for the baseline. Optionally add BI Engine for the dashboards. Not "just switch the ETL to batch priority" (that doesn't fix the cost or isolation) and not "add more projects" (on-demand slots aren't guaranteed).

TL;DR · DOMAIN 5

- On-demand = pay per TiB scanned. Editions = pay for slots, with reservations, baseline + autoscaling, commitments. Flat-rate is retired.
- Isolate workloads with reservations/assignments. Use batch priority for non-urgent queries.
- Composer: idempotent, parameterized DAGs, sensors, retries, backfills.
- Monitoring: Monitoring (metrics/alerts) · Logging (sinks, log-based metrics) · INFORMATION_SCHEMA.JOBS · billing export.
- Quota/rate errors → exponential backoff. Failure → multi-region, replicas, snapshots, seek/replay, dead-letter, idempotent reprocessing.

USING YOUR SENIOR'S FILES

Old name → new name

The ExamTopics dump is from 2022 and the notes from 2019. Use this table whenever an old name shows up, and answer with the modern product in mind.

Old name (in dumps)	Today	Note
Cloud Datalab	Vertex AI Workbench / Colab Enterprise	Datalab was retired
Cloud ML Engine / AI Platform	Vertex AI	Training, prediction, pipelines all live there
Stackdriver (Logging, Monitoring, Trace...)	Cloud Logging, Cloud Monitoring, Cloud Trace	Google Cloud Observability
Data Studio	Looker Studio	
Cloud Data Catalog	Dataplex Universal Catalog	Catalog merged into Dataplex
Analytics Hub	BigQuery sharing	Same idea: exchanges, listings, linked datasets
Cloud DLP	Sensitive Data Protection (DLP API)	
Cloud Datastore	Firestore in Datastore mode	
Cloud Functions	Cloud Run functions	
BigQuery flat-rate	BigQuery Editions (reservations, autoscaling)	Flat-rate was retired in 2023
BigQuery legacy streaming (insertAll)	Storage Write API	Legacy still works, but it's not the recommended answer
Legacy SQL	GoogleSQL (Standard SQL)	Always answer with GoogleSQL
Pub/Sub Lite	Pub/Sub or Managed Service for Apache Kafka	Lite has been shut down
Dataproc SOCKS proxy for UIs	Component Gateway	
Preemptible VMs	Spot VMs	Same idea, no 24-hour limit
Cloud Dataprep by Trifacta	Dataprep (Alteryx Designer Cloud)	Rarely tested now
Transfer Appliance / Storage Transfer Service	Unchanged	Still very testable
gsutil	gcloud storage	gsutil still works

HOW TO TRUST THE DUMP

- The "correct answer" shown on ExamTopics pages is often wrong. The community discussion (highest-voted answer) is usually closer to right, but not always.
- When the dump's answer conflicts with the 6 golden rules or with this guide, trust the rules. That's what we'll do together in the quiz sessions.
- Questions about retired products (Datalab, ML Engine CUSTOM tier, Legacy SQL details) are low priority. Learn the concept, not the trivia.

SPEED ROUND

If you see this → pick that

60 trigger phrases. Cover the right column and test yourself. This is the fastest way to revise.

Question says...	Answer leans to...
Real-time ingestion of events at any scale, decouple producers/consumers	Pub/Sub
Load Pub/Sub messages into BigQuery with no transformation	BigQuery subscription
Duplicate messages from Pub/Sub	Ack within the deadline / dedupe on a unique ID
Reprocess messages from yesterday after a bug	Pub/Sub seek to timestamp/snapshot
Messages for one customer must stay in order	Pub/Sub ordering keys
Unified batch + streaming, serverless, autoscaling	Dataflow (Apache Beam)
Late-arriving data, out-of-order events	Event time + watermark + allowed lateness
Group user activity until they go idle	Session windows
Moving average over last hour, updated every minute	Sliding windows
Stop a streaming job without losing data	Drain
Deploy new code to a running streaming job, keep state	Update (`--update`), compatible change
Enrich every event with a small lookup table	Side input
Bad records shouldn't fail the pipeline	Dead-letter output/table
Non-developers launch a pipeline with parameters	Dataflow (Flex) template
Migrate existing Spark/Hadoop with minimal changes	Dataproc
Spark without managing clusters	Serverless for Apache Spark
Cut Dataproc cost	Ephemeral clusters + GCS + Spot secondary workers + autoscaling
Install a library on every Dataproc node	Initialization actions
No-code ETL, many connectors	Cloud Data Fusion
SQL transformations with version control and tests	Dataform
Orchestrate multi-step pipelines with dependencies and retries	Cloud Composer
Lightweight serverless chain of API calls	Workflows
Trigger something when a file lands in GCS	Eventarc → Cloud Run / functions (or a Composer sensor)
Replicate DB changes into BigQuery in near real time	Datastream (CDC)
Migrate MySQL/Postgres to managed DB with minimal downtime	Database Migration Service

Hundreds of TB, slow network	Transfer Appliance
Recurring copy from S3/Azure/on-prem to GCS	Storage Transfer Service
Migrate Teradata/Redshift, or load Google Ads data	BigQuery Data Transfer Service
Petabyte analytics in SQL, serverless	BigQuery
Query data in S3/Azure without moving it	BigQuery Omni / BigLake
Reduce bytes scanned on date-filtered queries	Partitioning (+ require partition filter)
Speed up filters on customer_id within partitions	Clustering
Avoid joins between orders and line items	Nested & repeated fields
Dashboards need sub-second response	BI Engine
Same aggregation queried over and over	Materialized view
Restore a table someone overwrote this morning	Time travel (`FOR SYSTEM_TIME AS OF`)
Key lookups, <10 ms, IoT/time series, PB scale	Bigtable
Bigtable one node overloaded	Row-key hotspot → redesign key (Key Visualizer)
Separate Bigtable batch analytics from serving traffic	Replication + app profiles (single-cluster routing)
Global relational DB, strong consistency, 99.999%	Spanner
Regional MySQL/Postgres, moderate size	Cloud SQL
Postgres, more performance + analytics on live data	AlloyDB
Mobile app documents, offline sync	Firestore
Sub-ms caching / leaderboard	Memorystore
Data must not be deleted for 7 years	GCS retention policy + Bucket Lock
Rarely accessed archive, cheapest	GCS Archive class + lifecycle rules
Control/rotate/revoke encryption keys	CMEK (Cloud KMS)
Find and mask PII	Sensitive Data Protection (DLP)
Join on PII without exposing it	Deterministic tokenization / FPE
Prevent data exfiltration even with stolen creds	VPC Service Controls
Data must stay in region X	Regional resources + resource-locations org policy
Share query results, not the source tables	Authorized view
Each manager sees only their region's rows	Row-level access policy
Hide/mask a sensitive column for some users	Policy tags + dynamic data masking
Share live datasets with partners, no copies	Analytics Hub (BigQuery sharing)
Catalog, data quality, lineage across lake + warehouse	Dataplex
Analysts build ML models with SQL	BigQuery ML
OCR, image labels, sentiment, translation, no ML team	Pre-trained APIs (Vision, NL, Translation, Document AI)
Model great on train, bad on test	Overfitting → regularization / dropout / more data
Chatbot that answers from company documents	RAG: embeddings + vector search + Gemini
ETL slows dashboards; costs unpredictable	Editions reservations for ETL vs BI (+ autoscaling)

Quota / rate-limit errors	Exponential backoff
Which queries cost the most?	INFORMATION_SCHEMA.JOBS / admin resource charts

PDE one-page cheat sheet

Rules: managed > DIY · least ops · meet *all* requirements, then cheapest · least privilege · fix the cause

Storage picker

- **GCS** files/lake · Std / Nearline 30d / Coldline 90d / Archive 365d
- **BigQuery** analytics, PB, SQL, seconds
- **Cloud SQL** regional OLTP ≤ ~64 TB, scale up
- **AlloyDB** fast Postgres + columnar (HTAP)
- **Spanner** global OLTP, horizontal, 99.999%
- **Bigtable** row-key NoSQL, <10 ms, IoT, ≥1 TB
- **Firestore** app documents, offline sync
- **Memorystore** sub-ms cache

BigQuery design

- Partition by date (+ require filter) · cluster ≤4 cols
- Denormalize: nested/repeated · no SELECT * · LIMIT ≠ cheaper
- Load: Avro best, batch loads free · stream: Storage Write API
- Time travel ≤7d · snapshots · clones · long-term after 90d
- External / federated (EXTERNAL_QUERY) · BigLake · Omni

Bigtable

- Only index = row key · no timestamp-first keys
- Field promotion > salting · reverse ts for newest
- Hotspot → Key Visualizer → redesign key
- SSD↔HDD can't switch · app profiles: multi-cluster (HA) vs single-cluster (isolation)

Spanner / SQL

- Spanner PK: UUIDv4, not sequential · interleave children
- Cloud SQL: HA standby (zone) · cross-region replica (DR) · read replicas

Pub/Sub

- At-least-once (exactly-once option) · ordering keys
- Duplicates = late acks · DLQ topic · seek to replay
- 1 subscription per app needing all msgs
- BigQuery / GCS subscriptions = no-code sink

Dataflow / Beam

- PCollection · ParDo/DoFn · GroupByKey · Combine · CoGroupByKey · Flatten
- Side input = small lookup · tagged outputs = dead letter
- Windows: fixed · sliding · session · global
- Event time + watermark · allowed lateness · triggers (watermark / processing-time / count) · accumulating vs discarding
- Drain = stop safely · Update = redeploy with state · Snapshots
- Hot key → withHotKeyFanout · Flex Templates · FlexRS for cheap batch

Dataproc

- Lift & shift Spark/Hadoop · GCS not HDFS
- Ephemeral per job · Spot secondaries · autoscaling
- Init actions · Metastore · Component Gateway
- Serverless Spark = no cluster

Other tools

- Data Fusion = no-code ETL · Dataprep = visual cleaning
- Dataform = SQL ELT + assertions + git
- Composer = Airflow DAGs · Workflows = API chains · Scheduler = cron · Eventarc = events
- Datastream = CDC → BQ · DMS = DB migration
- STS = online bulk copy · Appliance = offline huge · BQ DTS = SaaS/warehouse → BQ
- 1 TB @100 Mbps ≈ 1 day

Security

- Predefined roles → groups/SAs · avoid basic roles
- Analyst = dataViewer (dataset) + jobUser (project)
- CMEK = you control/revoke · EKM = keys outside Google · CSEK only GCS/GCE
- DLP: redact (lost) · tokenize/FPE (joinable, reversible) · hash (one-way)
- VPC-SC = anti-exfiltration · org policy = residency
- Audit logs: Admin always on · BQ data access on by default
- Separate projects dev / test / prod

Analysis & sharing

- BI Engine = in-memory · MV = auto-refresh aggregates
- Authorized view = results only · row access policy = rows · policy tags + masking = columns
- Analytics Hub = share without copying · clean rooms
- Looker = governed metrics (LookML) · Looker Studio = quick dashboards

ML

- Overfit → L1/L2, dropout, more data · underfit → more features/complexity
- Imbalanced → precision/recall/PR-AUC · recall = catch them all
- High cardinality → embeddings / hash buckets
- SQL users → BQML · common task → APIs · custom easy → AutoML · full control → Vertex custom
- Online = low latency · batch = big async
- RAG: chunk → embed → vector index → retrieve → Gemini

Ops & cost

- On-demand $/TiB · Editions = slots, reservations, baseline + autoscale, commitments
- Isolate ETL vs BI reservations · batch priority for non-urgent
- Max bytes billed · custom quotas · dry run · billing export
- 429/quota → exponential backoff
- DR: multi-region · replicas · snapshots · seek/replay · idempotent reruns

You're ready for the quizzes.

Next up: domain-by-domain quiz sets built from your senior's 244 questions (answers checked and updated), extra questions for the new topics, then timed 50-question mock exams.

Facts checked against Google's current exam guide (5 sections, Nov 2026). Product names change often, so double-check the official exam guide before booking.